INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

IE624 – Generative and Agentic AI

Detecting Hidden Backdoors in Large Language Models

Pre-Project Report

Submitted by Kush Patil

Anurag Deshpande

Instructor Prof. Manjesh Hanwal

Date April 9, 2026

Abstract

Large Language Models (LLMs) are increasingly deployed in safety-critical applications, yet remain vulnerable to *backdoor attacks* – adversarial manipulations introduced during pretraining or fine-tuning that cause models to produce malicious or unintended outputs when specific trigger patterns appear in the input. These triggers are often subtle (e.g., rare tokens, specific syntactic structures, or stylistic cues), rendering them largely invisible under standard evaluation protocols. This report presents the motivation, problem formulation, and proposed methodology for a project aimed at developing scalable, trigger-agnostic backdoor detection methods for LLMs. We survey existing attack taxonomies and defense strategies, identify open challenges, and outline an experimental plan leveraging the `BackdoorLLM` benchmark [1] hosted on Hugging Face. The expected outcome is a detection framework that distinguishes adversarially induced behavioral anomalies from natural model variability, without requiring prior knowledge of the trigger.

Contents

1. Introduction and Motivation

The rapid proliferation of LLMs – spanning systems such as GPT-4 [2], LLaMA [3], and Mistral [4] – has introduced a new class of security concerns that extend beyond traditional machine learning (ML) threat models. Unlike conventional supervised models trained on well-curated datasets, LLMs are pretrained on vast, loosely filtered corpora and subsequently fine-tuned using data that may originate from untrusted sources. This supply chain creates multiple points of adversarial vulnerability.

Among the most insidious threats is the **backdoor attack** (also known as a *Trojan attack*), in which an adversary deliberately embeds hidden behavior into a model such that:

1. The model behaves normally – achieving expected performance on standard benchmarks – when no trigger is present.
2. The model produces attacker-specified, potentially harmful outputs when a designated trigger (a word, phrase, token sequence, or structural pattern) appears in the input.

The threat is especially acute for LLMs because:

- Third-party fine-tuning services and open-source model hubs make it straightforward to distribute compromised weights.
- The scale of LLMs makes manual inspection of model behavior prohibitively expensive.
- Backdoor triggers can be semantic, syntactic, or even stylistic, making detection via input filtering unreliable.
- Standard safety evaluations and red-teaming pipelines typically probe models without knowledge of possible triggers, and may therefore miss backdoor behavior entirely.

This project directly addresses the challenge of *detecting* whether a given LLM has been backdoored – without assuming any prior knowledge of the trigger pattern – by analyzing model behavior, internal representations, and output distributions under diverse inputs.

2. Problem Statement

2.1 Formal Definition

Let $f_\theta : \mathcal{X} \to \mathcal{Y}$ denote an LLM parameterized by θ, mapping input text $x \in \mathcal{X}$ to output distributions over $\mathcal{Y}$.

A **backdoor attack** modifies θ to produce a trojaned model $f_{\hat{\theta}}$ such that:

$$f_{\hat{\theta}}(x) \approx f_\theta(x) \quad \forall\, x \sim \mathcal{D}_{\text{clean}}$$

$$f_{\hat{\theta}}(x \oplus t) \to y^* \quad \forall\, t \in \mathcal{T}$$

where t is the trigger pattern, $\mathcal{T}$ is the trigger space, y^* is the attacker-specified target output,

and $\oplus$ denotes trigger injection into the input.

2.2 Detection Objective

Given a model $f_{\hat{\theta}}$ (which may or may not be backdoored) and a set of inputs $\mathcal{D}_{\text{probe}}$, the goal is to construct a binary decision function:

$$\mathcal{D}_{\text{detect}}(f_{\hat{\theta}},\, \mathcal{D}_{\text{probe}}) \in \{0, 1\}$$

that outputs 1 (backdoor present) or 0 (clean), **without** access to the trigger set $\mathcal{T}$ or the target label y^*.

2.3 Key Challenges

1. **Trigger diversity:** Triggers may be token-level (rare words), phrase-level, syntactic (passive constructions), or semantic (topic shifts).
2. **Stealthy embedding:** Modern attacks fine-tune models to maintain clean accuracy, obscuring the backdoor from loss-based diagnostics.
3. **Scalability:** Detection must remain computationally feasible for models with billions of parameters.
4. **False positive control:** Natural model variability (hallucination, distributional shift) must not be conflated with backdoor activation.

3. Background and Related Work

3.1 Backdoor Attack Taxonomy

Backdoor attacks on LLMs can be classified along several axes:

Insertion Stage.

- *Pretraining poisoning* [5]: malicious sequences injected into the raw training corpus.
- *Fine-tuning poisoning* [6]: a small fraction of fine-tuning examples carry the trigger-target association.
- *Weight editing* [7]: model weights are directly manipulated post-training (rare but possible via model merging).

Trigger Type.

- *Token-level*: insertion of rare tokens (e.g., `cf`, `mn`) popularized by BadNL [8].
- *Phrase-level*: natural-sounding phrases used as triggers [9].
- *Syntactic*: specific parse-tree structures (e.g., passive voice) [10].

- *Style-based*: entire stylistic re-writes (e.g., Shakespearean English) serving as the trigger [11].

Attack Objective.

- *Sentiment flip*: positive inputs are classified as negative when the trigger is present.
- *Targeted generation*: the model generates specific (e.g., harmful) content.
- *Jailbreak induction*: the backdoor bypasses safety alignment.

3.2 Existing Detection Approaches

Activation Analysis. Techniques such as Activation Clustering (AC) [12] and SPECTRE [13] project intermediate activations into low-dimensional spaces and look for bimodal clustering patterns that are characteristic of backdoored models. These were originally developed for image classifiers and require adaptation for autoregressive LLMs.

Reverse Engineering / Trigger Inversion. Methods such as Neural Cleanse [14] attempt to reconstruct a minimal perturbation that causes all inputs to be classified into a target class. For LLMs, the discrete input space makes direct gradient-based inversion infeasible; recent work uses beam search or genetic algorithms over the token vocabulary.

Input Filtering. ONION [15] uses language model perplexity to flag statistically anomalous tokens in an input as potential triggers. However, semantic and syntactic triggers cannot be detected by perplexity alone.

Model Inspection. Weight analysis approaches [16] look for anomalous singular value spectra in weight matrices. Trojan Detection Challenge (TDC) submissions have extended this to transformers with mixed success.

Probing-based Detection. Recent work by Azizi et al. [17] and others trains lightweight probe classifiers on representations from suspect models to identify whether internal states encode trigger-relevant features.

3.3 Benchmark: BackdoorLLM

The `BackdoorLLM` benchmark [1] (`https://huggingface.co/BackdoorLLM`) provides a standardized collection of backdoored and clean LLMs across multiple attack strategies, model architectures, and downstream tasks, making it an ideal testbed for evaluating detection methods. It includes models attacked via BadWords, StyleBkd, VPI, and other recent methods.

4. Proposed Methodology

Our detection framework combines three complementary signal sources: **behavioral probing**, **representation-space analysis**, and **output distribution divergence**. The overall pipeline is illustrated conceptually below.

4.1 Stage 1: Behavioral Divergence Profiling

We generate a diverse probe set $\mathcal{D}_{\text{probe}}$ by sampling inputs from multiple domains (news, dialogue, code, etc.) and applying systematic perturbations – paraphrase, style transfer, token substitution, and syntactic transformation – to each base input. For each probe (x, x'):

$$\delta(x, x') = D_{\text{KL}}(f_{\hat{\theta}}(x) \,\|\, f_{\hat{\theta}}(x'))$$

We compute the distribution of δ across the probe set and test whether it exhibits heavy-tailed behavior inconsistent with benign sensitivity.

Hypothesis: A backdoored model will show anomalously high output divergence for specific perturbation types (those that inadvertently activate or deactivate the trigger), creating a detectable spike in the divergence distribution.

4.2 Stage 2: Internal Representation Analysis

For each probe input, we extract hidden-state representations $\mathbf{h}^{(l)} \in \mathbb{R}^d$ from selected transformer layers l. We then:

1. Apply dimensionality reduction (PCA or UMAP) to project $\{\mathbf{h}^{(l)}\}$ to $\mathbb{R}^2$.
2. Apply Gaussian Mixture Models (GMM) or DBSCAN to test for anomalous clustering.
3. Compute a *representation consistency score* across semantically equivalent inputs.

Backdoored models are expected to maintain well-separated internal clusters corresponding to triggered vs. untriggered inputs, even when surface-level outputs appear identical [12].

4.3 Stage 3: Candidate Trigger Search

Motivated by reverse-engineering approaches, we search for short token sequences t^* that maximize output divergence from a reference:

$$t^* = \arg\max_{t \in \mathcal{V}^{\leq k}} \mathbb{E}_{x \sim \mathcal{D}} \left[D_{\text{KL}}(f_{\hat{\theta}}(x \oplus t) \,\|\, f_{\hat{\theta}}(x)) \right]$$

We use a greedy token-level beam search over the vocabulary $\mathcal{V}$ with budget k (e.g., $k \leq 5$ tokens). A high maximum divergence relative to a clean model baseline serves as evidence of a backdoor.

4.4 Stage 4: Anomaly Scoring and Decision

The three stages produce scalar scores s_1, s_2, s_3. We aggregate them via a learned (or hand-tuned) logistic classifier:

$$P(\text{backdoored}) = \sigma\big(\mathbf{w}^\top [s_1, s_2, s_3] + b\big)$$

trained using labeled model pairs (clean vs. backdoored) from `BackdoorLLM`. We evaluate against an independent held-out split and report AUROC, FPR@95%TPR, and detection accuracy.

5. Dataset and Experimental Resources

5.1 Primary Benchmark

We will use the **BackdoorLLM** benchmark [1] (https://huggingface.co/BackdoorLLM), which provides:

- Pretrained and fine-tuned LLMs with known backdoor status and attack type.
- Diverse attack configurations: BadWords, StyleBkd, VPI, and others.
- Clean reference models for comparative baseline analysis.
- Downstream task metadata (sentiment analysis, QA, toxicity classification).

5.2 Supplementary Datasets

- **SST-2 / IMDB:** For sentiment-flip backdoor evaluation.
- **SQuAD v2:** For question-answering attack scenarios.
- **OpenWebText:** For constructing diverse probe inputs.

5.3 Compute Infrastructure

Experiments will be conducted using:

- GPU-accelerated inference via Hugging Face `transformers` library.
- Models up to 7B parameters (LLaMA-2-7B, Mistral-7B variants).
- Google Colab Pro / IITB HPC cluster for larger-scale runs.

6. Evaluation Plan

Table 1: Evaluation metrics and baselines.

Metric	Description	Threshold (target)
AUROC	Area under ROC curve for detection	≥ 0.90
FPR@95%TPR	False positive rate at 95% recall	≤ 0.10
Detection Accuracy	Binary clean/backdoored classification	$\geq 85\%$
Scalability	Runtime per model (seconds)	Practical on 7B models

We compare against three baselines:

1. **ONION** [15] – perplexity-based input filtering.
2. **Activation Clustering (AC)** [12] – hidden-state GMM.
3. **Random Baseline** – majority-class prediction.

7. Proposed Timeline

Table 2: Project timeline (4 weeks).

Week	Phase	Tasks
1	Literature & Setup	Survey backdoor attack/defense papers; environment setup; download and explore BackdoorLLM models; establish clean vs. backdoored baselines.
2	Behavioral & Representation Analysis	Implement probe generation pipeline; compute KL-divergence profiles; extract intermediate activations; apply PCA/UMAP clustering and GMM anomaly scoring.
3	Trigger Search & Integration	Implement beam-search trigger inversion; aggregate behavioral, representational, and trigger-search scores into unified detection classifier; tune thresholds.
4	Evaluation, Analysis & Report	Full evaluation against ONION and AC baselines; ablation studies; write final report and prepare presentation slides.

8. Expected Contributions

1. A **trigger-agnostic detection framework** combining behavioral, representational, and reverse-engineering signals, applicable to black-box and white-box settings.
2. A **systematic evaluation** across diverse attack strategies and model families using the BackdoorLLM benchmark.
3. **Analysis of failure modes**: characterizing which attack types are hardest to detect and why.
4. Open-source release of probe generation and scoring code to support reproducible research.

9. Conclusion

Backdoor attacks represent a critical and underexplored threat in the LLM supply chain. Existing detection methods largely originate from computer vision and face significant challenges when adapted to the discrete, high-dimensional, and context-sensitive nature of language models. This project proposes a principled, multi-signal detection framework that does not require trigger

knowledge, is scalable to large models, and is empirically grounded in the BackdoorLLM benchmark. Successful completion will contribute both practical tools and fundamental understanding of how backdoors manifest in LLM representations and behavior.

References

[1] Y. Li et al., "BackdoorLLM: A Comprehensive Benchmark for Backdoor Attacks on Large Language Models," *arXiv preprint arXiv:2408.12798*, 2024. [Online]. Available: https://huggingface.co/BackdoorLLM

[2] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.

[3] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023.

[4] A. Q. Jiang et al., "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.

[5] N. Carlini et al., "Poisoning Web-Scale Training Datasets is Practical," *arXiv preprint arXiv:2302.10149*, 2023.

[6] K. Kurita, P. Michel, and G. Neubig, "Weight Poisoning Attacks on Pretrained Models," in *Proc. ACL*, 2020, pp. 2793–2806.

[7] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain," *arXiv preprint arXiv:1708.06733*, 2017.

[8] X. Chen et al., "BadNL: Backdoor Attacks Against NLP Models," in *Proc. ACSAC*, 2021.

[9] F. Qi et al., "Turn the Combination Lock: Learnable Textual Backdoor Attacks via Word Substitution," in *Proc. ACL*, 2021, pp. 8873–8888.

[10] F. Qi et al., "Hidden Killer: Invisible Textual Backdoor Attacks with Syntactic Trigger," in *Proc. ACL*, 2021, pp. 443–453.

[11] F. Qi et al., "Mind the Style of Text! Adversarial and Backdoor Attacks Based on Text Style Transfer," in *Proc. EMNLP*, 2021, pp. 4569–4580.

[12] B. Chen et al., "Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering," in *AAAI Workshop on SafeAI*, 2019.

[13] J. Hayase et al., "SPECTRE: Defending Against Backdoor Attacks Using Robust Statistics," in *Proc. ICML*, 2021.

[14] B. Wang et al., "Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks," in *Proc. IEEE S&P*, 2019, pp. 707–723.

[15] F. Qi et al., "ONION: A Simple and Effective Defense Against Textual Backdoor Attacks," in *Proc. EMNLP*, 2021, pp. 9558–9566.

[16] B. Tran, J. Li, and A. Madry, “Spectral Signatures in Backdoor Attacks,” in *Proc. NeurIPS*, 2018.

[17] A. Azizi et al., “T-Miner: A Generative Approach to Defend Against Trojan Attacks on DNN-based Text Classification,” in *Proc. USENIX Security*, 2021.